

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación

Práctica de Laboratorio Número 2

(Algoritmos de ordenamientos Quick Sort y Merge Sort)

Profesor:
José Canache
Arquitectura del Computador

Autor:
Ronald Montilla
C.I. 32.011.453

6 de agosto de 2026

Índice

1. Registros y sus principales diferencias.	2
1.1. Registros temporales	2
1.2. Registros Guardados	2
1.3. Registros de Argumentos	3
1.4. Registros de Retorno	3
2. Impacto de registros frente memoria en el rendimientos de los algoritmos de ordenamiento implementados.	3
2.1. Ciclos de reloj	3
2.2. Frecuencia de operaciones	4
2.3. Riesgos de datos de carga	4
3. Impacto de las estructuras de control e instrucciones de salto en la eficiencia de algoritmos MIPS32	4
4. Complejidades computacionales de QuickSort, MergeSort y sus implicaciones en MIPS32	5
4.1. Comparación de Complejidad Computacional	5
4.2. Implicaciones en la Implementación para MIPS32	5
5. Fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32	6
6. Análisis de la Implementación en MARS	6
6.1. Instrucciones predominantes (R, I, J)	6
6.2. Ventajas del modelo RISC de MIPS en algoritmos de ordenamiento	7
6.3. Uso del modo de ejecución paso a paso (Step / Step Into) en MARS	7
6.4. Herramientas de MARS útiles para observar registros y detectar errores lógicos	7
6.5. Camino de datos para una instrucción tipo R en MARS	7
6.6. Camino de datos para una instrucción tipo I en MARS	8
7. ¿Cuál de los dos algoritmos es mejor?	8
8. Análisis y discusión de los resultados	8
9. Código de los algoritmos.	9
9.1. Quick Sort:	9
9.2. Merge Sort	11

1. Registros y sus principales diferencias.

MIPS32 posee 32 registros de 32 bits cada uno. Alguno de los principales registros son:

1.1. Registros temporales

\$t0 - \$t9 (Registro 8-15, 24-25): Conocidos como *Registros Temporales*, se utilizan sin necesidad de guardar sus datos, es decir, se deben emplear teniendo en cuenta que al llamar otras funciones sus datos se pueden perder a menos que sean importantes en cuyo caso deben ser respaldados. En las implementaciones de quick sort y merge sort en este laboratorio fueron indispensables ya que se usaron principalmente para realizar operaciones matemáticas de calculo de indices, almacenamiento temporal del resultado de comparar datos y como variables contadoras. Un ejemplo de esto se puede visualizar a continuación:

```
1 sll $t0, $a2, 2
2 add $t0, $t0, $a0
3 lw $t1, 0($t0) # $t1 = pivot
4 add $t2, $a1, $zero # $t2 sera mi variable i para recorrer el
   arreglo.
5 add $t3, $a1, $zero # $t3 sera mi variable j y apuntara al
   elemento despues del ultimo menor.
```

En esta porción del código de Quick Sort se emplearon los registros temporales para calcular el índice del arreglo, alojar un elemento del arreglo y crear variables contadoras que posteriormente fueron empleadas en un bucle for.

1.2. Registros Guardados

\$s0 - \$s7 (registros 16 - 23): Mejor conocidos como *Registros Guardados*, son registros que deben ser preservados en un función, es decir, sus datos son importante para la función, por lo cual deben ser guardados al momento de realizar llamadas a otras funciones y deben retornar su valor anterior al momento de finalizar dicha función.

En la implementación de los algoritmos de ordenamiento fueron cruciales para alojar datos importantes para el funcionamiento de los algoritmos. Por ejemplo, en Merge Sort fueron empleados para alojar condiciones de parada para ciclos, datos empleados constantemente en toda la función y datos que debían ser resguardados porque funciones externas lo podían modificar(en caso de ser empleados en registros temporales):

```
1 slt $t2, $s0, $s1
2 beq $t2, $zero, finMergeSort
3 # Entra en un for:
4 add $s2, $a1, $zero # $s2 empezara en el primer elemento i = 0
5 addi $s3, $a2, -1
6 sub $s3, $s3, $s0 # $s3 sera la condicion de parada del for.
```

1.3. Registros de Argumentos

\$a0 - \$a3 (Registros 4-7): los conocidos como *Registros de Argumentos* se emplean para alojar los argumentos que serán usados por una función. Al momento de emplear una sub-rutina estos deben ser guardados y al finalizar restaurados, como se hizo en las sub-rutinas del algoritmo de Quick Sort y Merge Sort del presente laboratorio.

En este ejemplo se muestran los 4 registros(\$a0 - \$a3) de argumentos que fueron recibidos por la sub-rutina merge para su posterior uso, Pero antes fueron guardados en la pila ya que estos datos deben ser preservados:

```
1 merge:
2     # $a0: direccion del arreglo, $a1: indice al primer elemento.
3     # $a2: indice al medio del arreglo, $a3: indice del ultimo
4     # elemento.
5     addi $sp, $sp, -20
6     sw $ra, 16($sp)
7     sw $a0, 12($sp)
8     sw $a1, 8($sp)
9     sw $a2, 4($sp)
10    sw $a3, 0($sp)
```

1.4. Registros de Retorno

\$v0 - \$v1 (Registros 2-3): Estos registros son los encargados de alojar los resultados devueltos por una sub-rutina. En el caso de la implementación de Quick Sort fueron empleados para alojar el resultado devuelto por la sub-rutina merge, la cual debía retornar el índice de partición.

2. Impacto de registros frente memoria en el rendimientos de los algoritmos de ordenamiento implementados.

La elección del uso de registros o memoria en los algoritmos de ordenamientos tienen claros impactos en el rendimiento de un programa en MIPS32. En donde la jerarquía de memoria y el pipeline radican directamente en estos impactos. Algunos de los principales impactos de esta elección son los siguientes:

2.1. Ciclos de reloj

Registros \$t0 - \$t9 y \$s0 - \$s7: estos registros se encuentran alojados en el banco de registros de la CPU por lo que el acceso a ellos toma 1 ciclo de reloj, la lectura y escritura sobre estos registros se hace directamente en el banco de registros, sin latencia de bus.

Memoria principal lw y sw: el acceso a la memoria principal es complejo debido a la jerarquía de memoria, en donde se debe acceder a la memoria L1, si el dato se encuentra suele tomar alrededor de 2 y 4 ciclos de reloj, pero si no se encuentra se debe acceder a la jerarquía inferior y la penalización puede ser de decenas o cientos de ciclos de reloj.

Es aquí donde se puede observar claramente la diferencia de rendimiento a la hora de emplear registros o recurrir a la memoria principal. Por lo cual siempre es recomendable recurrir cuanto menos a la memoria en los algoritmos de ordenamiento, ya que suelen realizar repetidamente las mismas operaciones.

2.2. Frecuencia de operaciones

Los algoritmos de ordenamiento suelen realizar una y otra vez las mismas operaciones como comparar o intercambiar datos. Si se tienen dichos datos en la memoria principal el procesador debe obtener esos datos, compararlos, fusionarlos o cambiarlos de posición y volver a almacenarlos en la memoria, por lo cual, ante grandes cantidades de datos esto suele requerir más ciclos de reloj de lo esperado.

2.3. Riesgos de datos de carga

El uso continuo de la memoria principal(\$lw, \$sw) en este tipo de algoritmos suele mantener al procesador constantemente detenido(*stall*), principalmente por los riesgos de datos de carga, es decir, fallos que se producen cuando una instrucción requiere de un dato que aun no esta cargado porque que, en el *pipeline* aun no se encuentra disponible. Como por ejemplo en la siguiente instrucción:

```
1 lw $t0, 0($s1)
2 slt $t1, $t0, $t2
```

Operar directamente con registros anula estos fallos que detienen al procesador mientras espera el dato solicitado, evitando el desperdicio de ciclos de reloj.

Por todo lo antes mencionado, los registros suelen presentar mejor rendimiento para los algoritmos de ordenamiento en MIPS32.

3. Impacto de las estructuras de control e instrucciones de salto en la eficiencia de algoritmos MIPS32

El impacto de las estructuras de control en MIPS32 está determinado directamente por cómo interactúan con su arquitectura segmentada (pipeline de 5 etapas: IF, ID, EX, MEM, WB) y la jerarquía de memoria. Los saltos y bucles mal optimizados degradan el rendimiento al generar penalizaciones por ciclo.

En la presente implementación de quickSort y mergeSort las estructuras de control fueron muy empleadas, por la naturaleza de los algoritmos de ordenamiento de recorrer los elementos. Estas estructuras favorecen la localidad temporal al emplear repetidamente las mismas instrucciones, sin embargo, también puede tener un impacto negativo en la eficiencia de los algoritmos con ciertas instrucciones como las de acceso a memoria(como \$lw, \$sw que generan latencia en el bus) y continuos riesgos de datos. También, anidar muchos bloques suele alterar la eficiencia de un algoritmo al aumentar su complejidad temporal.

Uno de los principales impactos se producen por el pipeline debido a que instrucciones de salto condicional(como `beq` y `bne`) deben realizar una comparación antes de realizar el

salto, esta comparación se produce en la etapa *EX* mientras que el procesador ejecuta las instrucciones posteriores. Si la condición se cumple se produce el salto, pero el procesador se encuentra procesando las instrucciones que estaban luego de la instrucción de salto condicional por lo que ahora deberá deshacer todo lo hecho, malgastando de esta manera ciclos de reloj.

Por otro lado, instrucciones de salto incondicional(*j*, *jr*, *jal*) pueden alterar la eficiencia de los algoritmos por aspectos como: saltos a direcciones muy lejanas que pueden provocar penalización por fallos debido a que las instrucciones no se encuentran directamente en la cache, por otro lado, si el salto es corto es muy probable que las instrucciones siguientes a ese salto ya se encuentren en la cache principal, favoreciendo la localidad espacial y la tasa de aciertos. Es por ello, que establecer bloques de instrucciones no muy grandes es esencial en este tipo de casos para reducir la penalización por fallos y favorecer la localidad espacial.

4. Complejidades computacionales de QuickSort, MergeSort y sus implicaciones en MIPS32

La diferencia fundamental entre Quicksort y Mergesort radica en la garantía de tiempo de ejecución frente al consumo de memoria auxiliar, lo que condiciona de forma directa la gestión de la pila y los registros en un entorno de bajo nivel como MIPS32.

4.1. Comparación de Complejidad Computacional

Tiempo en el mejor caso y caso promedio: Ambos logran una complejidad de $O(n \log n)$. Sin embargo, Quicksort suele ser más rápido en la práctica debido a menores factores constantes en sus operaciones elementales.

Tiempo en el peor caso: Mergesort garantiza $O(n \log n)$ de forma determinista sin importar el orden inicial. Quicksort degrada a $O(n^2)$ si la selección del pivote es desfavorecida (por ejemplo, al procesar arreglos ya ordenados con un pivote en el extremo).

Espacio auxiliar (Memoria): Quicksort opera de forma in-place (sobre el mismo arreglo), requiriendo únicamente $O(\log n)$ de espacio en la pila para las llamadas recursivas en el caso promedio. Mergesort requiere $O(n)$ de espacio adicional en memoria para crear arreglos temporales durante la fase de mezcla.

Estabilidad: Mergesort es estable (mantiene el orden relativo de elementos con claves iguales), mientras que la implementación estándar de Quicksort no lo es.

4.2. Implicaciones en la Implementación para MIPS32

Quicksort: Resulta muy eficiente en la pila de MIPS32. Cada llamada recursiva solo necesita salvar en la pila el registro de retorno *\$ra* y los registros guardados *\$s0-\$s7* que utilice la función. Con una profundidad de recursión promedio de $O(\log n)$, la pila se mantiene ligera.

Realiza las particiones e intercambios directamente sobre el arreglo original manipulando datos cargados en registros temporales *\$t0-\$t9*. Esto genera una excelente localidad espacial que minimiza fallos de caché y reduce al mínimo el uso de punteros.

Mergesort: Exige reservar un buffer auxiliar de $O(n)$ palabras en la memoria de datos (RAM). En ensamblador MIPS32 bare-metal, esto implica gestionar asignación dinámica mediante llamadas al sistema (syscall con servicio sbrk) o reservar un bloque estático de tamaño máximo previo, complicando la arquitectura del código.

La etapa de mezcla (merge) requiere copiar elementos continuamente del arreglo original al auxiliar y viceversa. Esto incrementa drásticamente la densidad de instrucciones de carga (lw) y almacenamiento (sw), sobrecargando la memoria y potencialmente causando fallos(stalls) en el pipeline de MIPS.

5. Fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32

Las fases del camino de datos de MIPS32 son cinco(IF, ID, EX, MEM, WB):

1. Búsqueda de Instrucción(IF): Se libera la dirección por parte del contador del programa(PC) que se ingresa en la memoria de instrucciones para su búsqueda. El contador de programa se incrementa(PC + 4).

2. Decodificación de Instrucción(ID): la memoria de instrucciones libera la instrucción que será procesada para obtener los registros necesarios, los datos de los registros y las señales de control.

3. Ejecución(EX): Fase en la que ALU recibe sus operandos, la señal de control de la operación(ALUOP) y realiza la operación indicada con sus operandos. Se calculan direcciones de salto condicional o el desplazamiento para accesos a memoria base más desplazamiento.

4. Acceso a memoria(MEM): La memoria recibe indicaciones por parte de señales de control para escribir o leer en ella. Las instrucciones aritméticas o lógicas simplemente reenvían el resultado sin modificar la memoria.

5. Escritura en los registros(WB): Fase en la que se escribe en el registro de escritura el dato obtenido de la ALU o la memoria de datos.

6. Análisis de la Implementación en MARS

6.1. Instrucciones predominantes (R, I, J)

En el desarrollo de esta práctica predominaron fuertemente las instrucciones de tipo **I (Inmediato)** y **R (Registro)**. Las de tipo I (como **lw**, **sw**, **addi**, **beq**, **bne**) fueron fundamentales, ya que los algoritmos de ordenamiento requieren interactuar constantemente con la memoria para leer y escribir los elementos de los arreglos, además de ser la base para manejar las condiciones de parada en los bucles. Por otro lado, las instrucciones de tipo R (como **add**, **slt**, **sll**) se utilizaron continuamente para el cálculo de las direcciones de memoria (como al multiplicar índices por 4 usando corrimientos lógicos), las comparaciones entre elementos y la aritmética básica de nuestros índices. Las instrucciones de tipo J (**j**, **jal**) también fueron vitales para la recursividad y los saltos entre subrutinas, pero en términos de volumen, las I y R hicieron el trabajo pesado dentro de las iteraciones.

6.2. Ventajas del modelo RISC de MIPS en algoritmos de ordenamiento

El modelo RISC de MIPS nos ofrece una gran ventaja gracias a su arquitectura de carga y almacenamiento (*Load/Store*) y su amplio banco de registros. Al implementar algoritmos básicos como Quick Sort y Merge Sort, que realizan múltiples operaciones repetitivas e intercambios, tener 32 registros nos permitió mantener nuestros contadores, índices y valores temporales directamente en la CPU. Esto resulta sumamente eficiente, ya que evitamos accesos innecesarios y costosos a la memoria principal. Además, al tener un repertorio de instrucciones de tamaño fijo y sencillo, el mapeo de la lógica abstracta de lenguajes de alto nivel a código ensamblador resultó ser mucho más directo y predecible.

6.3. Uso del modo de ejecución paso a paso (Step / Step Into) en MARS

El modo de ejecución paso a paso fue nuestra principal herramienta de depuración (*debugging*). Utilizando las funciones *Step* y *Step Into*, pudimos verificar el comportamiento del código instrucción por instrucción. Esto nos sirvió especialmente para confirmar que el cálculo de las direcciones de los arreglos (el desplazamiento de bytes) fuera el correcto antes de ejecutar un `lw` o `sw`. También fue crucial al usar *Step Into* en las llamadas recursivas de Quick Sort, lo que nos permitió observar de cerca cómo se comportaba el puntero de la pila (`$sp`) y asegurar que los registros importantes se estaban guardando y restaurando en el orden adecuado sin corromperse.

6.4. Herramientas de MARS útiles para observar registros y detectar errores lógicos

Definitivamente, el panel lateral derecho de **Registros (Registers)** combinado con la ventana del **Segmento de Datos (Data Segment)** fueron las herramientas más útiles. El panel de registros nos permitía ver en tiempo real cómo se actualizaban nuestras variables iteradoras (`$t0-\t9`) y confirmar que las subrutinas devolvieran los valores esperados. Por su parte, el Segmento de Datos nos ayudó enormemente a detectar errores lógicos; por ejemplo, si calculábamos mal un índice, podíamos visualizar físicamente en la cuadrícula de la memoria que estábamos sobrescribiendo datos en posiciones incorrectas del arreglo, permitiéndonos ajustar el código rápidamente.

6.5. Camino de datos para una instrucción tipo R en MARS

Para visualizar el camino de datos en MARS, se puede utilizar la herramienta **MIPS X-Ray** (ubicada en el menú *Tools*). Al animar una instrucción tipo R, como un `add`, podemos ver visualmente cómo la instrucción pasa por las 5 etapas: primero, el contador de programa (PC) señala a la memoria para extraer la instrucción (IF); luego, en la decodificación (ID), las líneas de datos se iluminan mostrando cómo se extraen los dos operandos desde el banco de registros. Seguidamente, esos datos viajan a la ALU para realizar la suma (EX). Como es una instrucción aritmética, se ignora el bloque de memoria de datos (MEM) y el resultado viaja directamente a través del multiplexor de regreso al banco de registros para ser guardado (WB).

6.6. Camino de datos para una instrucción tipo I en MARS

Utilizando la misma herramienta **MIPS X-Ray**, el camino de datos para una instrucción tipo I (como **lw**) cambia su recorrido. Tras la búsqueda (IF) y decodificación (ID), observamos que solo se lee un registro fuente. El segundo operando es el valor inmediato de 16 bits que pasa por la unidad de extensión de signo para convertirse en 32 bits. Ambos entran a la ALU (EX) para sumarse y calcular la dirección de memoria efectiva. A diferencia de las tipo R, en este punto el camino de datos entra en el bloque de la **Memoria de Datos** (MEM), indicando una operación de lectura. Finalmente, el valor extraído de esa dirección viaja de vuelta al banco de registros para ser escrito en el registro destino (WB).

7. ¿Cuál de los dos algoritmos es mejor?

Tanto Quick Sort como Merge Sort emplean la estrategia "divide y vencerás", sin embargo, en la práctica se emplea uno en su forma recursiva (Quick Sort) y el otro en su forma iterativa (Merge Sort). A la hora de elegir entre uno u otro se deben tomar en cuenta diferentes aspectos.

En primer lugar la complejidad temporal. Ambos tienen una complejidad promedio de $O(n \log n)$, sin embargo en el peor de los casos Quick Sort puede degradarse a la complejidad $O(n^2)$ mientras Merge Sort se mantiene en la misma complejidad. Por otro lado, Quick Sort suele ser la preferida para procesar grandes cantidades de datos a altas velocidades mientras que Merge Sort suele requerir mayor consumo de memoria debido a la creación de estructuras auxiliares de elementos. Además, algo a tomar en cuenta es que Merge Sort no suele modificar la posición relativa de los elementos, mientras que el otro algoritmo sí lo hace.

Por todo lo antes mencionado llegamos a la siguiente conclusión. Si buscas velocidad y menor consumo de memoria elige Quick Sort, pero si buscas estabilidad posicional y de tiempo debes irte por Merge Sort.

8. Análisis y discusión de los resultados

En el momento de elaborar el código de ambos algoritmos se logró observar que la complejidad al elaborar el algoritmo de ordenamiento Merge Sort de forma iterativa es mucho superior a elaborarlo de forma recursiva, su complejidad radica principalmente en el cálculo de los índices y prevención de desbordamientos. Esto se pudo notar durante la ejecución del algoritmo de ordenamiento Merge Sort donde los problemas con índices fue constante y se tuvo que elegir la mejor solución.

Gracias a las herramientas del entorno MARS MIPS32 se pudo avanzar con mayor rapidez al detectar errores con las herramientas de MARS y observar el flujo de los datos y actualización de los registros. También se pudo contabilizar el número de instrucciones de cada tipo durante la ejecución de cada algoritmo (con la herramienta "instruction counter"). En el algoritmo de ordenamiento Merge Sort se contabilizaron un total de 969 instrucciones, 415(42%) de tipo R, 486(50%) de tipo I y 68(7%) de tipo J. En el algoritmo de ordenamiento Quick Sort se contabilizaron un total de 1278 instrucciones, 506(39%) de tipo R, 681(53%) de tipo I y 91(7%) de tipo J.

Para las pruebas de ambos códigos se emplearon arreglos de diferentes tamaños y desordenados de diferentes formas, cuando ocurría un error se buscaba inmediatamente su solución. De esta se llegó a la conclusión correcta.

9. Código de los algoritmos.

El presente laboratorio se basa en familiarizar por parte de estudiante con la sintaxis y el entorno de trabajo MARS a través de la resolución de problemas clásicos. En este caso, los problemas se basan en algoritmos de ordenamiento: Quick Sort y Merge Sort. Los siguientes códigos muestran la implementación de estos algoritmos en MIPS32, para los cuales se implemento uno recursivo(quick sort) y otro iterativo(merge sort).

9.1. Quick Sort:

Es uno de los algoritmos de ordenamiento más famosos. Fue creado en la década de los 60 y es sumamente famoso por su rapidez a la hora de ordenar grandes cantidades de datos. Se basa en el principio de divide y vencerás ya que divide repetidamente los elementos tomando un *pivot* como referencia para colocar los elementos menores a la izquierda y los mayores a la derecha, de esta manera el *pivot* queda en su posición correcta.

Este algoritmo tiene una complejidad temporal promedio de $O(n \log(n))$ al tomar el *pivot* de manera aleatoria pero su complejidad aumenta al tomar el *pivot* de manera incorrecta, como cuando ya están ordenados los elementos, a $O(n^2)$.

Implementación en MIPS32:

```
1 swap:
2     # $a0: direccion del primer elemento, $a1: direccion del
3     #     segundo elemento
4     lw $t0, 0($a0)
5     lw $t1, 0($a1)
6     sw $t1, 0($a0)
7     sw $t0, 0($a1)
8     jr $ra
9
10 partition:
11     # $a0 corresponde a la direccion del arreglo "A[]". $a1
12     #     corresponde al indice del primer elemento
13     # $a2 corresponde al indice del ultimo elemento.
14     addi $sp, $sp, -16
15     sw $ra, 12($sp)
16     sw $a2, 8($sp)
17     sw $a1, 4($sp)
18     sw $a0, 0($sp)
19     sll $t0, $a2, 2
20     add $t0, $t0, $a0
21     lw $t1, 0($t0) # $t1 = pivot
22     add $t2, $a1, $zero # $t2 sera mi variable i para recorrer el
23     #     arreglo.
24     add $t3, $a1, $zero # $t3 sera mi variable j y apuntara al
25     #     elemento despues del ultimo menor.
```

```

14   forPartition:
15       slt $t4, $t2, $a2
16       beq $t4, $zero, finPartition
17       sll $t4, $t2, 2
18       add $t4, $t4, $a0
19       lw $t5, 0($t4) # $t5 = A[i]
20       slt $t6, $t1, $t5
21       beq $t6, $zero, intercambio
22       addi $t2, $t2, 1
23       j forPartition
24   intercambio:
25       # Se intercambian los elementos A[i] y A[j]
26       sll $t6, $t3, 2
27       add $t6, $t6, $a0
28       add $a0, $t4, $zero
29       add $a1, $t6, $zero
30       # Como swap modifica $t0 y $t1:
31       add $t6, $t0, $zero
32       add $t7, $t1, $zero
33       jal swap
34       # Recuperacion de datos importantes:
35       add $t0, $t6, $zero
36       add $t1, $t7, $zero
37       lw $a0, 0($sp)
38       lw $a1, 4($sp)
39       #Incremento de variables controladoras:
40       addi $t2, $t2, 1
41       addi $t3, $t3, 1
42       j forPartition
43   finPartition:
44       add $v0, $t3, $zero # $v0 retornara el indice de
45           particion
46       sll $t3, $t3, 2
47       add $t3, $t3, $a0
48       add $a0, $t0, $zero
49       add $a1, $t3, $zero
50       jal swap
51       lw $a0, 0($sp)
52       lw $a1, 4($sp)
53       lw $a2, 8($sp)
54       lw $ra, 12($sp)
55       addi $sp, $sp, 16
56       jr $ra

```

```

1 quickSort:
2     # $a0 corresponde con la direccion del arreglo A[], $a1
3     # corresponde al indice del primer elemento
4     # $a2 corresponde al indice del ultimo elemento
5     addi $sp, $sp, -20
6     sw $ra, 16($sp)
7     sw $a0, 12($sp)
8     sw $a1, 8($sp)
9     sw $a2, 4($sp)
10    slt $t0, $a1, $a2
11    beq $t0, $zero, finQuickSort
12    jal partition
13    sw $v0, 0($sp)
14    addi $a2, $v0, -1
15    jal quickSort
16    lw $v0, 0($sp)
17    lw $a2, 4($sp)
18    addi $a1, $v0, 1
19    jal quickSort
20    finQuickSort:
21    lw $a2, 4($sp)
22    lw $a1, 8($sp)
23    lw $a0, 12($sp)
24    lw $ra, 16($sp)
25    addi $sp, $sp, 20
26    jr $ra

```

9.2. Merge Sort

Otro de los más famosos y eficientes algoritmos de ordenamiento, empleando la estrategia de *divide y vencerás* al dividir una lista de elementos hasta que queda uno solo (que ya está ordenado) y luego los fusiona dejando cada elemento en su lugar correspondiente. Una de sus características es que al momento de ordenar elementos iguales no los cambia de posición, esta característica es muy importante en muchas implementaciones. Mantiene una complejidad temporal en todos los casos de $O(n \log n)$ y una complejidad espacial de $O(n)$ debido a la creación de arreglos auxiliares.

En esta implementación se emplearon dos funciones principales *mergeSort* (que divide los elementos de la lista a fusionar), *min* (que retorna el menor entre dos enteros) y *merge* (que fusiona esos elementos seleccionados). El código es el siguiente:

```

1 merge:
2     # $a0: direccion del arreglo, $a1: indice al primer elemento.
3     # $a2: indice al medio del arreglo, $a3: indice del ultimo
      elemento.
4     addi $sp, $sp, -20
5     sw $ra, 16($sp)
6     sw $a0, 12($sp)
7     sw $a1, 8($sp)
8     sw $a2, 4($sp)
9     sw $a3, 0($sp)
10    add $t0, $a1, $zero # primer indice de la primera mitad del
      arreglo "i".
11    addi $t1, $a2, 1 # primer indice de la segunda mitad del
      arreglo "j".
12    li $t2, 0 # indice actual del arreglo auxiliar. "k".
13    addi $t3, $a3, 1
14    subu $t3, $t3, $a1 # $t3: numero de elementos del arreglo
15    add $t4, $a0, $zero
16    # Arreglo auxiliar "arr2" con tamano igual al numero de
      elementos del arreglo pasado por argumentos.
17    # Su direccion se guarda en $t9
18    la $t9, arr2
19    add $a0, $t4, $zero
20    whileMerge:
21        slt $t4, $a2, $t0
22        bne $t4, $zero, whileDosMerge
23        slt $t4, $a3, $t1
24        bne $t4, $zero, whileUnoMerge
25        # Primera comparacion y guardado:
26        sll $t4, $t0, 2
27        add $t4, $t4, $a0
28        lw $t4, 0($t4) # $t4 = A[i]
29        sll $t5, $t1, 2
30        add $t5, $t5, $a0
31        lw $t5, 0($t5) # $t5 = A[j]
32        sll $t7, $t2, 2
33        add $t7, $t7, $t9
34        addi $t2, $t2, 1
35        slt $t6, $t4, $t5
36        beq $t6, $zero, guardarSegundo
37        sw $t4, 0($t7)
38        addi $t0, $t0, 1
39        j whileMerge
40        guardarSegundo:
41            sw $t5, 0($t7)
42            addi $t1, $t1, 1
43            j whileMerge
44    whileUnoMerge:
45        slt $t4, $a2, $t0
46        bne $t4, $zero, reemplazoArreglo
47        sll $t4, $t0, 2

```

```

48     add $t4, $t4, $a0
49     lw $t4, 0($t4) # $t4 = A[i]
50     sll $t7, $t2, 2
51     add $t7, $t7, $t9
52     addi $t2, $t2, 1
53     sw $t4, 0($t7)
54     addi $t0, $t0, 1
55     j whileUnoMerge
56 whileDosMerge:
57     slt $t4, $a3, $t1
58     bne $t4, $zero, reemplazoArreglo
59     sll $t5, $t1, 2
60     add $t5, $t5, $a0
61     lw $t5, 0($t5) # $t5 = A[j]
62     sll $t7, $t2, 2
63     add $t7, $t7, $t9
64     addi $t2, $t2, 1
65     sw $t5, 0($t7)
66     addi $t1, $t1, 1
67     j whileDosMerge
68 reemplazoArreglo:
69     add $t0, $a1, $zero # indice del primer elemento del
70                          arreglo.
71     li $t1, 0 # indice del primer elemento del arreglo
72                          auxiliar.
73 forMerge:
74     slt $t2, $a3, $t0
75     bne $t2, $zero, finMerge
76     sll $t3, $t1, 2
77     add $t3, $t3, $t9
78     lw $t3, 0($t3) # $t3 = Au[k]
79     addi $t1, $t1, 1
80     sll $t4, $t0, 2
81     add $t4, $t4, $a0
82     sw $t3, 0($t4)
83     addi $t0, $t0, 1
84     j forMerge
85 finMerge:
86     lw $a3, 0($sp)
87     lw $a2, 4($sp)
88     lw $a1, 8($sp)
89     lw $a0, 12($sp)
90     lw $ra, 16($sp)
91     addi $sp, $sp, 20
92     jr $ra

```

```

1 min:
2   # Recibe dos numeros enteros y retorna el menor.
3   # $a0: numero a, $a1: numero b, $v0 salida del numero menor
4   slt $t1, $a0, $a1
5   beq $t1, $zero, retornarB
6   add $v0, $a0, $zero
7   j finMin
8   retornarB:
9       add $v0, $a1, $zero
10  finMin:
11      jr $ra

```

```

1 mergeSort:
2   # $a0: direccion del arreglo, $a1: indice al primer elemento.
3   # $a2: valor correspondiente al ultimo indice + 1
4   addi $sp, $sp, -32
5   sw $s3, 28($sp)
6   sw $s2, 24($sp)
7   sw $s1, 20($sp)
8   sw $s0, 16($sp)
9   sw $ra, 12($sp)
10  sw $a2, 8($sp)
11  sw $a1, 4($sp)
12  sw $a0, 0($sp)
13  li $s0, 1 # Numero de elementos * 2 a fusionar (numElements)
14  move $s1, $a2 # $t1 = total de elementos.
15  whileMergeSort:
16      slt $t2, $s0, $s1
17      beq $t2, $zero, finMergeSort
18      # Entra en un for:
19      add $s2, $a1, $zero # $s2 empezara en el primer elemento
20      i = 0
21      addi $s3, $a2, -1
22      sub $s3, $s3, $s0 # $s3 sera la condicion de parada del
23      for.
24      forMergeSort:
25          slt $t4, $s3, $s2
26          bne $t4, $zero, incrementoWhileMergeSort
27          # Calculo los argumentos para la funcion merge:
28          sll $a0, $s0, 1
29          add $a0, $a0, $s2
30          addi $a0, $a0, -1
31          addi $a1, $a2, -1
32          jal min
33          add $a2, $s0, $s2
34          addi $a2, $a2, -1 # $a2 sera el indice medio del
35          subarreglo.
36          add $a3, $v0, $zero # $a3 sera el min ($s1 + ($s0 * 2) -
37          1, n - 1) y ultimo indice del subarreglo.
38          lw $a0, 0($sp)
39          move $a1, $s2 # $a1 sera el primer indice del subarreglo.

```

```

36     jal merge
37     lw $a0, 0($sp)
38     lw $a1, 4($sp)
39     lw $a2, 8($sp)
40     sll $t4, $s0, 1
41     add $s2, $s2, $t4 # Incremento para el for.
42     j forMergeSort
43     incrementoWhileMergeSort:
44         sll $s0, $s0, 1
45         j whileMergeSort
46 finMergeSort:
47     lw $a0, 0($sp)
48     lw $a1, 4($sp)
49     lw $a2, 8($sp)
50     lw $ra, 12($sp)
51     lw $s0, 16($sp)
52     lw $s1, 20($sp)
53     lw $s2, 24($sp)
54     lw $s3, 28($sp)
55     addi $sp, $sp, 32
56     jr $ra
57 mostrarArreglo:
58     addi $sp, $sp, -12
59     sw $ra, 8($sp)
60     sw $a0, 4($sp)
61     sw $a1, 0($sp)
62     # $a0 es la direccion del vector A[], $a1 es el indice del
        ultimo elemento
63     li $t0, 0
64     addi $t3, $a1, 1
65     loop:
66         slt $t2, $t0, $t3
67         beq $t2, $zero, finLoop
68         sll $t1, $t0, 2
69         add $t1, $t1, $a0
70         add $t2, $a0, $zero
71         li $v0, 11
72         li $a0, 32
73         syscall
74         lw $a0, 0($t1)
75         li $v0, 1
76         syscall
77         add $a0, $t2, $zero
78         addi $t0, $t0, 1
79         j loop
80     finLoop:
81         lw $a1, 0($sp)
82         lw $a0, 4($sp)
83         lw $ra, 8($sp)
84         addi $sp, $sp, 12
85         jr $ra

```